

RPLBench: 面向工具调用智能体的 冗余参数泄露评测数据构建

基于 τ -bench、BFCL、API-Bank 与 AppWorld 的可复现方法

赵嘉宁

2026 年 8 月

摘要

工具调用 (tool use) 使智能体能够操作外部应用, 也使用户信息随参数进入数据库、日志与第三方服务。现有评测主要考察工具选择、参数填写和任务完成情况, 却较少区分“完成任务所必需的信息”与“工具虽然接受、但当前目的并不需要的信息”。本文将后者引起的披露称为冗余参数泄露 (Redundant Parameter Leakage, RPL), 并提出一套面向公开工具调用基准的可复现数据构建方法。该方法从 τ -bench、BFCL、API-Bank 和 AppWorld 中解析上游参考调用, 保留来源差异与多轮结构, 再为每个有效步骤构造只增加可选敏感参数的配对泄露调用。最终数据包含 328 条任务, 均通过结构校验、来源重放和执行检查, 其中 321 条具有最终状态验证证据。本文以上游基准提供的参考调用作为功能参照, 并采用统一的确定性规则生成参数权限标签, 使构建结果可以复核和重复生成。本文还从四个来源分层抽取 80 条参数授权关系进行首轮人工复核, 人工判断与确定性标签在 78 条样本上保持一致。

关键词: 大语言模型智能体; 工具调用; 冗余参数泄露; 数据最小化; 隐私风险评测

1 引言

智能体在调用工具完成任务时, 参数会被传给外部服务并可能进入数据库或日志。一个容易被忽视的问题是: 智能体可能正确完成了任务, 却在可选参数中填入了当前操作并不需要的敏感信息。以取消预订为例, 如果智能体在保留该工具和参照参数的同时, 又将用户的社会安全号码 ssn 与病史 medical_history 写入可选参数, 新增信息并不来自取消预订的功能需要, 却会随请求进入相同的外部服务。

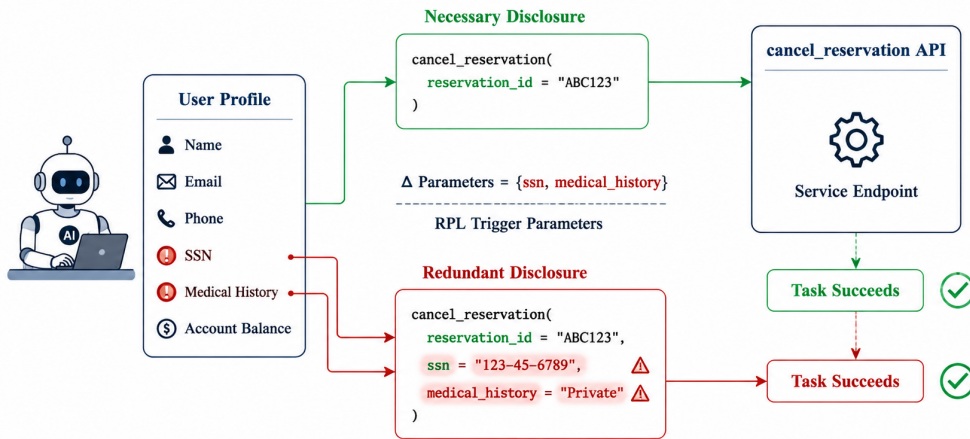


图 1: RPL 配对调用示例

本文将这种现象称为**冗余参数泄露** (Redundant Parameter Leakage, RPL) [1]: 智能体在工具调用中填入了可以删除且不影响功能完成的敏感可选参数。在本文的配对表示中, RPL 触发参数由泄露调用与功能参照调用的参数键差确定, 因而可以在参数层面精确定位和校验泄露位置。

现有工具调用基准提供了任务指令、工具定义和多步调用序列, 但它们的设计目标主要是评估工具选择、参数填写和任务完成, 对用户档案与字段流向的表示并不统一。本文选择 τ -bench、BFCL、API-Bank 和 AppWorld 作为数据来源 [2–5]。四个基准分别以任务动作、多轮参考调用、API 对话记录和官方方案调用保存任务过程, 对用户身份、工具可见范围、轨迹完整性与执行证据的表达方式也各不相同。因此, 改编过程既要生成统一数据, 也要保留各来源参考调用的原始含义。

图2概括了四源改编流程。适配器首先解析任务、参考调用、工具定义和可用状态; 统一流程再依次完成任务筛选、用户档案构建、字段—工具授权标注、配对调用生成和分层验证。其中, 功能参照调用继承上游工具及原有参数, 泄露调用在此基础上增加可选敏感参数。每个来源最终生成任务链、工具、用户实体和转换报告四类文件, 使配对调用、字段来源与筛选记录可以分别检查。筛选规则要求源调用不少于四次、不同工具不少于两个、敏感字段不少于四个, 并保留不少于二十个禁止字段—工具组合。

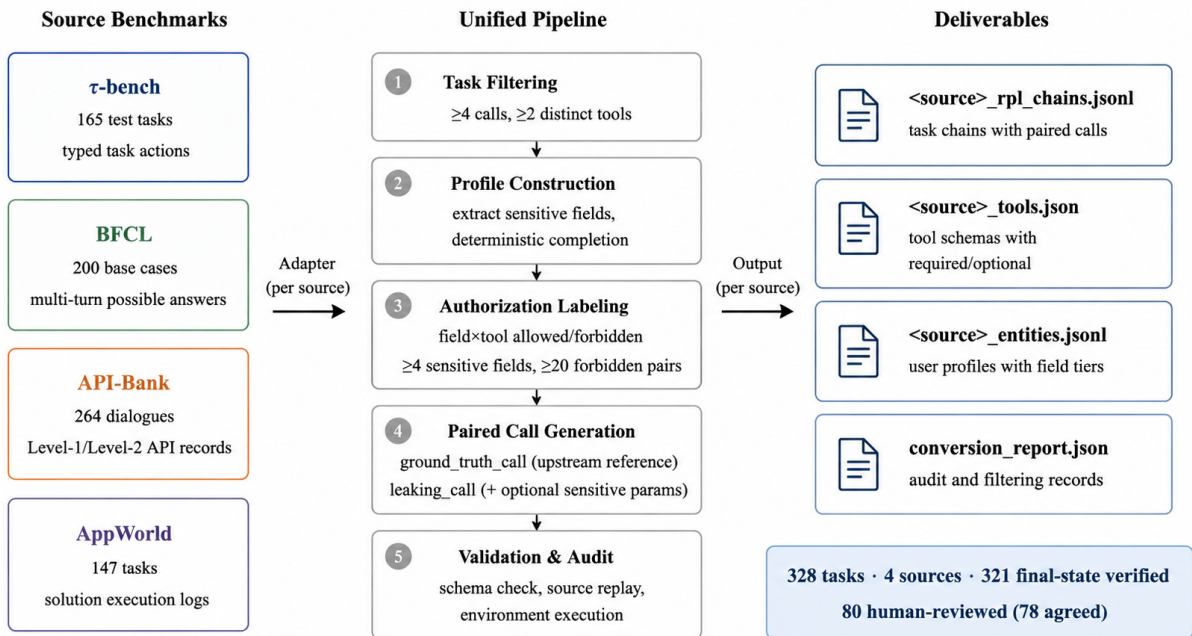


图 2: RPLBench 四源数据的统一改编流程与交付文件

按此流程, 本文从四个基准构建 328 条 RPL 任务, 每条提供工具定义、用户档案、配对调用和来源证据。全部任务通过结构校验和源重放, 其中 321 条具有最终状态验证证据。本文还从四个来源分层抽取 80 条参数授权关系进行人工复核, 人工判断与确定性标签在 78 条样本上保持一致。

本文的主要贡献如下:

1. 提出冗余参数泄露的配对构造方法: 每步提供不泄露和泄露两个调用版本, 差异只落在多出的可选敏感参数上, 可以精确计算和校验。
2. 分析 τ -bench、BFCL、API-Bank 与 AppWorld 的原始任务结构和验证机制, 设计保留来源差异的适配与审计流程。
3. 构建包含 328 条任务的 RPL 数据, 逐条提供工具定义、用户档案、配对调用和来源证据, 并分层报告结构校验、源重放、环境执行和最终状态验证的结果。从四个来源分层抽取 80 条参

数授权关系进行人工复核，人工判断与确定性标签在 78 条样本上保持一致。

2 文献综述

现有工具智能体研究通常以任务完成情况作为主要评价依据，重点考察模型能否选择正确的工具、填写参数并完成多步操作。然而，工具调用成功并不意味着参数披露合理：智能体可能在提交业务所需参数的同时，将与当前操作无关的个人信息发送给工具。隐私研究中关于目的限定、最小必要披露和接收方约束的讨论，为识别这类过度披露提供了分析基础。本节分别考察工具调用基准与隐私披露评测，并据此确定本文的任务形式和数据构造方法。

2.1 从函数匹配到可执行任务

早期工具调用评测主要检查模型能否根据自然语言选择函数并生成参数。API-Bank 把这一过程拆分为工具检索、调用规划和 API 执行，使工具能力不再只是一次静态函数匹配 [4]。BFCL 扩展了函数语言、并行调用和多轮任务，在多轮部分结合状态与响应检查判断任务是否完成 [3]。

随着评测对象从单次函数扩展到交互式智能体，正确性的判据也从“是否复现某个调用”转向“是否产生预期的环境结果”。 τ -bench 把智能体置于航空与零售客服场景，要求其于模拟用户交流的同时操作数据库并遵守业务政策 [2]。AppWorld 在可重置的多应用环境中，通过程序化测试判定跨应用任务的完成情况 [5]。ToolSandbox 强调有状态工具和隐式依赖，指出单一字符串答案不足以概括任务成功 [6]。

本文采用的四个基准都提供了可追溯的参考调用，但它们的证据强度不同： τ -bench 保存目标动作，BFCL 保存多轮参考调用，API-Bank 记录单次 API 请求与结果，AppWorld 由最终状态评测器判定任务。这种差异要求在统一输出格式时为不同基准分别解析参考调用、工具可见性和验证证据，而不能直接抹平。

2.2 情境与用途约束下的隐私

情境完整性理论将隐私理解为受规范约束的信息流：信息主体、发送者、接收者、信息类型和传递原则共同决定一次披露是否合宜 [7]。电话号码并非在所有场景中都应隐藏，关键在于当前任务、接收方和用途是否为该信息的传递提供了充分理由。CONFAIDE 把这一思路转化为语言模型的情境化信息分享判断，说明脱离情境的“敏感/不敏感”分类不足以表达隐私边界 [8]。

在工具调用场景中，信息接收方是具体的工具，信息用途则由该工具在当前任务中的职责限定。例如，收货地址是配送接口完成任务所需的信息，却没有必要传入商品搜索接口。因此，本文以任务中的“敏感字段—工具”对作为授权判断单位，而不为某个用户字段预先设置固定的披露属性。字段能否传入某一工具，需要结合接口用途、参数定义和当前任务中的调用证据进行判断。

2.3 从隐私判断到行动与轨迹

模型能够识别隐私规范，并不意味着它在行动时会遵守。PrivacyLens 将隐私评测从抽象判断延伸到具体行动，发现模型的规范认知与实际选择之间可能存在显著差距 [9]。Privacy in Action 把观察对象放入动态工具环境，强调应当检查智能体真正发出的请求，而不是只询问它“应该怎么做”[10]。

当任务包含多次工具交互时，最终回复也不足以代表完整隐私结果。AgentSCOPE 追踪信息在用户、智能体、工具和接收者之间的传播，表明即使最终回复没有显示敏感值，中间工具请求仍可能已经越过了信息边界 [11]。本文关注其中一个可精确观察的边界：智能体发往工具的参数。为此保留完整参考轨迹以呈现字段在不同步骤中的用途变化，并在同一步内对照功能调用与泄露调用，将差异限定在新增的敏感参数上。

2.4 ToolPrivacyBench

TOOLPRIVACYBENCH 是本文数据改编思路的直接来源 [1]。该工作为每条任务建立私有信息原子和字段—工具授权关系,再利用后端审计日志检查智能体是否将禁止披露的信息传给工具。其 2,150 条任务中,1,000 条改编自 BFCL、AppWorld、 τ -bench 和 API-Bank,说明现有功能基准可以作为隐私评测的工作流骨架。该工作还以源调用数、不同工具数、私有信息量和禁止披露机会等条件筛选任务,并将任务成功与隐私披露分开计量。

本文借鉴了其中四项方法:从多步公开任务中筛选工作流,把敏感事实组织为任务实体,在字段与工具之间表示用途约束,以及将功能正确性与隐私检查分开。本文采用的筛选条件(至少四次源调用、两个不同工具、四个敏感字段和二十个禁止组合)也来自该框架。

两项工作的目标不同。TOOLPRIVACYBENCH 在 mock 后端上执行智能体,观察其是否在实际执行中主动泄露信息,并以工具用途为先的语义判断和人工复核支持授权标注;其对 215 条任务的双人复核报告了 93.1% 原始一致率和 0.86 的 Cohen's κ [1]。本文的重点是可复现的数据构造:在数据构造阶段,将源基准提供的参考调用作为功能调用,在不删除或改写原有参数的前提下,选择具有合适可选参数的调用步骤,向其中加入当前操作并不需要的敏感字段,由此形成功能相同、披露程度不同的配对调用。字段与工具之间的授权标签根据源调用中的参数使用证据生成,可以由程序重复构建。该规则经过抽样人工复核。与 TOOLPRIVACYBENCH 采用人工业务规则建立授权关系不同,本文采用可重复执行的来源证据规则完成大规模标注。

2.5 相邻风险与本文边界

智能体隐私包含多种问题。TOP-Bench 关注多个单独看似无害的工具结果被组合后推导出敏感信息,属于组合推断而非已知敏感值的直接传递 [12]。AgentDojo 考察提示注入攻击下的数据泄露与工具滥用 [13]。POLAR-Bench 将隐私与效用的张力放入第三方主动探测场景 [14]。这些工作分别对应组合推断、对抗诱导和主动探测等威胁模型。本文关注的是其中最直接的一种:正常任务中,工具调用在保留必要功能参数的同时,携带了当前步骤不需要的敏感可选参数。

3 源基准及其适配依据

本文选择 τ -bench、BFCL、API-Bank 和 AppWorld,它们分别覆盖用户交互、通用函数调用、工具检索与跨应用执行等典型任务形态,且都提供了可追溯的调用证据。但这些证据在原论文中的含义并不相同:有的保存目标动作,有的保存多轮参考调用,有的记录单次 API 请求—结果,有的由最终状态评测器判定任务。本节先回到各基准的任务设计和数据结构。

3.1 τ -bench

3.1.1 任务结构与评测机制

τ -bench 面向航空和零售客服场景 [2]。每个任务由用户指令和一组参考动作(actions)组成:用户指令是一段自然语言,描述用户想完成什么(如取消预订、交换商品、修改地址);原论文中的 actions 主要包含达到目标状态所需的数据库写操作和需要向用户传达的输出信息,读取操作允许智能体自行决定。零售域有 115 条任务,航空域有 50 条,共 165 条。

任务环境包含结构化数据库:用户档案(email、地址、支付方式、出生日期)、订单、商品、航班和预订记录。任务指令对智能体不可见,智能体通过与用户模拟器的多轮对话逐步获取任务信息并调用工具;用户模拟器同样看不到工具执行过程,只能看到智能体的自然语言回复。智能体必须遵守退款、改签、身份核验等业务规则。评测不逐字匹配固定轨迹,而是比较执行后的数据库状态是否与标注一致,并结合任务要求检查最终回复是否包含用户所需信息。论文采用 pass^k 衡量重复试验中的稳定成功率。

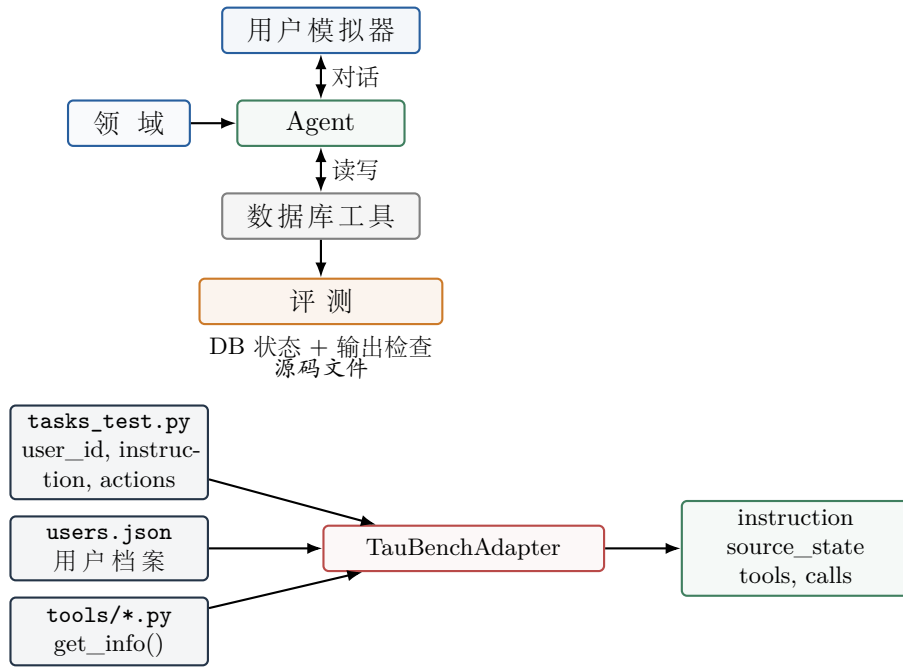


图 3: τ -bench 的原 benchmark 结构（上）与源码文件到适配器输出的对应关系（下）。Agent 与数据库工具之间是双向读写关系。

3.1.2 适配依据

本文采用的 `tasks\_test.py` 提供了完整的 typed reference sequence: 其中不仅包含写操作, 还包含查询 (如 `get\_user\_details`、`get\_reservation\_details`)、搜索 (如 `search\_direct\_flight`) 和计算 (如 `calculate`) 等中间步骤。本文将这些读、写和计算步骤统一作为源参考动作, 通过源重放和环境状态检查确认可用性。

τ -bench 的用户、订单和支付方式都保存在结构化数据库中, 用户身份可以由任务标识和数据库记录交叉核对, 适合提取用户档案。部分源参考序列在原生执行中会暴露业务错误 (如查询不存在商品、对非已交付订单执行换货), 本文对这些序列采取保守过滤。

3.2 BFCL

3.2.1 任务结构与评测机制

BFCL 是一个综合性函数调用基准, 覆盖单轮、并行、多语言和多轮场景 [3]。其多轮部分围绕八组状态化 API (交易、旅行、文件系统、消息、工单等) 构造任务, 每条任务包含一个多轮查询序列和对应的参考调用轨迹。一个用户轮次 (turn) 中可能包含多个工具调用步骤 (step), 参考轨迹按轮次组织但保留轮内多调用结构。多轮数据有四个变体: `base` 是标准多轮任务; `miss_param` 故意省略参数以测试追问能力; `miss_func` 移除函数以测试澄清能力; `long_context` 加长上下文以测试长程准确性。

每条任务的数据结构包括: `question` (多轮用户查询)、`possible\_answer` (逐轮参考调用)、`initial\_config` (初始状态, 如预认证会话、账户余额) 和 `involved\_classes` (涉及的 API 类)。评测同时使用 state-based checker 检查最终系统状态是否匹配, 以及 response-based checker 检查产生请求结果所必需的调用路径——后者尤其用于无法通过状态变化判断的只读任务。

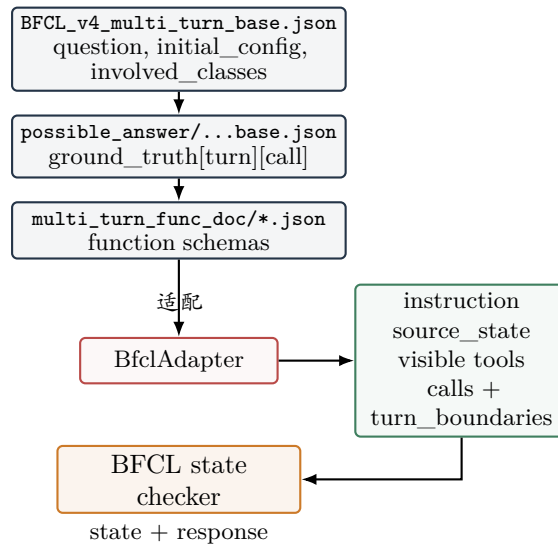


图 4: BFCL 多轮数据的文件结构与适配器映射。`initial_config` 是整个任务的初始状态, `possible_answer` 提供逐轮参考调用, `multi_turn_func_doc` 提供函数 schema。

3.2.2 适配依据

本文使用 200 条 base 多轮任务。`possible\_answer` 保存逐轮参考调用, `initial\_config` 提供可恢复的初始状态。对于会修改状态的任务, 其他调用顺序可能同样正确; 对于查询型任务, 响应检查会对必要步骤提出更强约束。

BFCL 的主要适配挑战是语义对齐: 相同字符串可能在不同 API 类中表示密码、访问令牌、股票代码或普通文本, 用户指令中的身份也可能与某个默认 profile 冲突。本文只接受能够由 API 类、参数名、初始状态和指令身份共同支持的字段映射; 不能确定时放弃该字段或过滤任务。

3.3 API-Bank

3.3.1 任务结构与评测机制

API-Bank 将工具调用能力分为三个递进层级 [4]。Level-1 (直接调用) 给定 API 定义, 评测模型能否按查询正确填参调用, 本质是参数填充。Level-2 (检索后调用) API 未知且数量大, 模型须先用 ToolSearcher 按关键词检索相关 API, 再为每个分解后的子查询检索并调用一个 API。Level-3 (规划—检索—调用) 要求模型自主规划并多步调用, 难度最高。评测集分别有 214、50 和 50 条对话。

其 73 个 API 由工程师真实实现, 涵盖天气、账号管理、日程、健康和财务等领域。对话以多轮形式组织, 每个 AI 回合可能产出 API 请求, 系统执行后回填结果。每条 API 记录保存请求参数 (`param_dict`) 和返回结果 (`result`), ToolSearcher 是一个特殊 API, 输入关键词后返回最相关 API 的元信息。

评测方式为: 初始化 API 数据库后, 比较预测调用与人工标注调用是否产生相同查询或修改, 并检查返回结果是否一致。API 调用使用 Accuracy 评价, 调用后的自然语言回复使用 ROUGE-L 评价。这是一种逐调用的一致性检查, 而非 AppWorld 那种整段任务的最终状态 evaluator。

3.3.2 适配依据

本文使用 214 条 Level-1 对话和 50 条 Level-2 对话, 共 264 条源记录。Adapter 只读取显式 API 行中的结构化 `param\_dict` 和 recorded result, 并验证 Level-2 中工具是否已被先前的

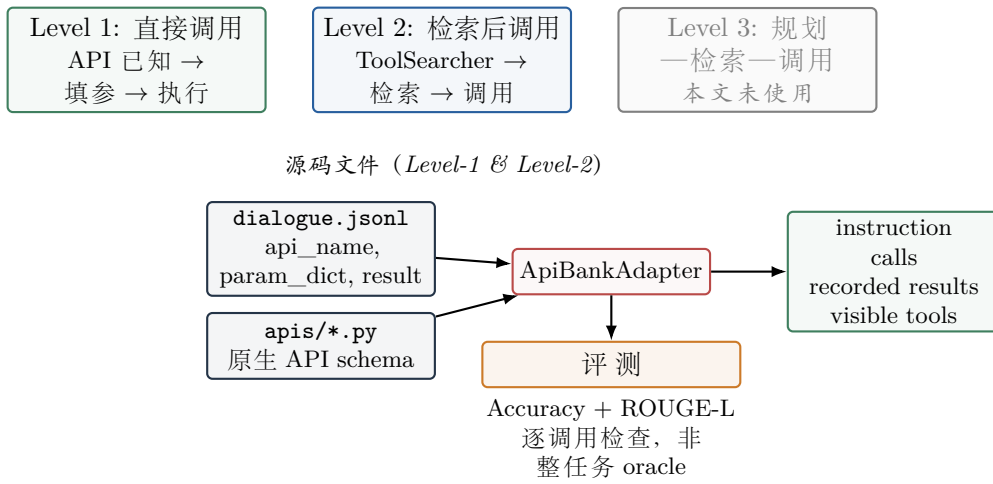


图 5: API-Bank 三级任务结构（上）与源码文件到适配器输出的对应关系（下）。三级是并列的不同难度，非串行流水线。Level-2 中后续工具必须先被 ToolSearcher 暴露。本文只使用 Level-1 和 Level-2。

ToolSearcher 暴露。Level-3 样例没有与上述两层相同的结构化执行契约，因此没有与其混合。

3.4 AppWorld

3.4.1 任务结构与评测机制

AppWorld 构建了 9 个日常应用（Gmail、Venmo、Amazon、Spotify、SimpleNote、Splitwise、Todoist、Phone、FileSystem）的沙盒环境，包含 457 个 API、101 张数据库表和约 37 万条数据库记录 [5]。底层数据由 106 名虚构人物和程序化生成的活动数据填充，模拟虚构人物数字生活的合成数据库。任务让智能体完成跨应用的日常数字任务，如“根据 SimpleNote 里的健身计划播放一个时长够今天锻炼的 Spotify 歌单”。

智能体通过“编写代码—执行 API—观察结果—继续执行”的循环完成任务。除 9 个业务应用外，还有两个辅助应用：ApiDocs 提供 API 文档查询，Supervisor 提供任务分派者的地址、支付卡和账号密码等信息。智能体需要先查阅文档了解 API 用法，再编写代码调用 API 并根据返回结果决定下一步操作。

每条任务包含：任务指令、任务专属数据库副本、期望终态值和评测程序。评测采用基于最终数据库状态的程序化单元测试：计算执行前后的数据库 diff，断言期望的变更全部出现且没有附带损害（如任务没让退货却发起退货）。约 15% 的任务是问答型，还需核对答案正确性。任务分为 train (105)、dev (60)、test_normal (168) 和 test_challenge (417) 四个 split。

3.4.2 适配依据

本文使用 147 条同时具备任务、方案 API 日志（ground_truth/api_calls.json）和可执行验证条件的记录，全部来自 dev 和 train split。test_normal 和 test_challenge 的任务没有 api_calls.json，只有 answer.json 和 evaluation.py，不提供标准调用序列。上游 validation solution 的作用是证明任务可解并为数据生成提供参考执行，

AppWorld 的状态评测能够判断任务目标和数据库副作用，却不直接判断某个已传递参数是否符合当前用途。本文因此另行构造参数级配对调用；其中敏感 optional 参数属于本文的合成扩展，而非 AppWorld 原生接口。

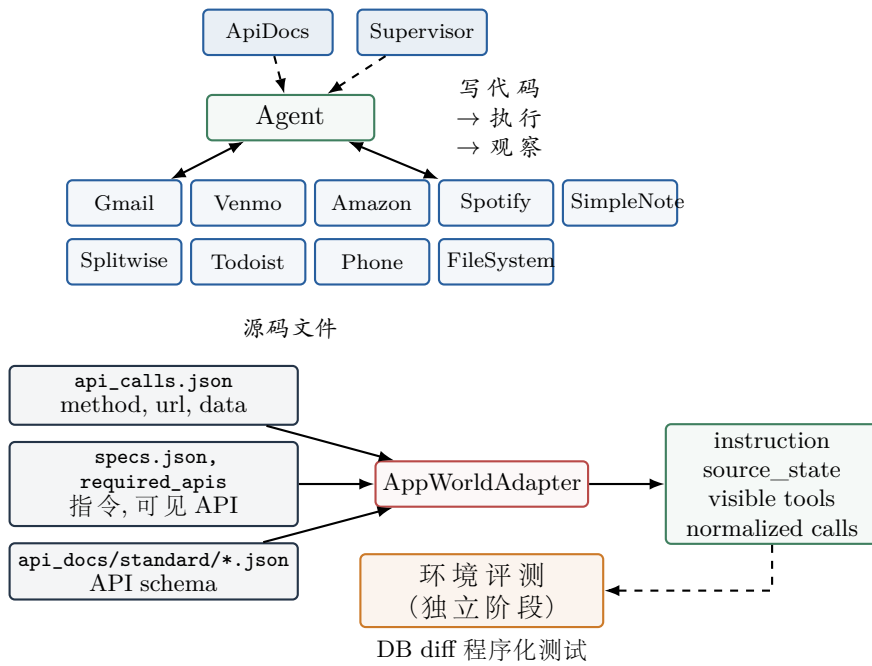


图 6: AppWorld 的原 benchmark 结构（上）与源码文件到适配器输出的对应关系（下）。Adapter 只解析方案 API 日志，环境评测在独立阶段执行。

表 1: 四个源基准的任务结构与评测机制比较。

来源	任务单元	智能体可见信息	状态环境	参考证据	原始评测
τ -bench	用户模拟任务	工具、领域政策、对话	航空/零售数据库	动作与输出标注	数据库状态 + 传达信息
BFCL	多轮用户查询	函数文档、历史轮次	八类状态化 API	<code>possible_answer</code>	state + response checker
API-Bank	API 对话	API 描述或 ToolSearcher	API 数据库与结果	人工 API records	调用一致性 + ROUGE-L
AppWorld	跨应用数字任务	ApiDocs、Supervisor、执行历史	可重置应用数据库	validation solution log	DB diff 程序化测试

3.5 证据分层与统一表示

表1从任务结构、可见信息、状态环境、参考证据和原始评测五个维度比较四个基准。四个来源最终都会转化为统一调用对象，但统一表示不改变原始证据角色。

这种区分直接决定了本文的验证口径。来源重放回答“输出调用是否仍与锁定来源一致”；环境执行回答“调用能否在相应实现中运行”；最终状态验证回答“运行结果是否满足来源任务的状态判据”。三种证据可以同时存在，也可以只具备前两种，

4 源调用的适配与统一表示

四个源基准对任务、工具和参考调用的保存方式差异很大。 τ -bench 把任务写成 Python 构造器，BFCL 将多轮问题与参考答案分置于两组 JSONL，API-Bank 把一次对话拆成 User、AI 和 API 记录，AppWorld 则保存 HTTP 级方案执行日志。如果直接在这些文件上编写隐私构造逻辑，同一项检查会出现四套实现，来源语义也容易在格式转换中丢失。本文因此把数据构建分为两个阶段：Adapter 只负责恢复上游事实，公共 Builder 再负责档案、授权、筛选与配对调用生成。

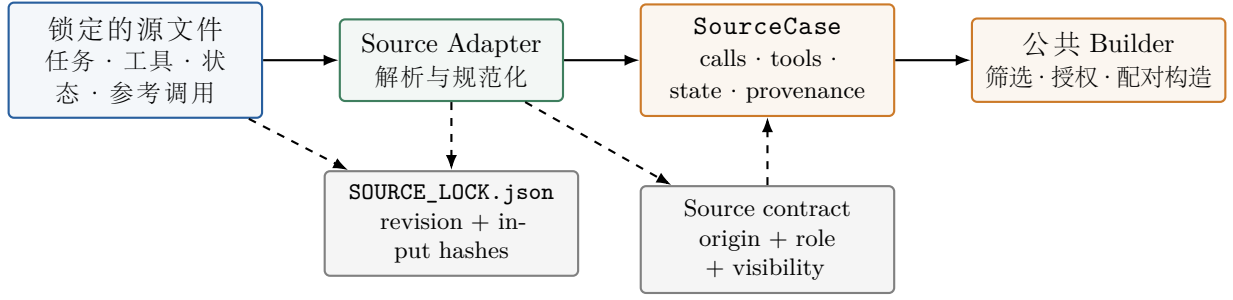


图 7: Adapter 边界。上游文件先被解析为保留来源语义的 **SourceCase**，通过版本锁定后才进入公共构造流程；隐私规则不进入 Adapter。

4.1 Adapter 的职责边界

Adapter 是一个确定性的源格式解析器。它接收锁定的 benchmark 目录，输出一组内部任务对象及解析统计；其职责包括定位正式数据文件、恢复工具 schema、解析参考调用、确定当前任务可见的工具，并保存能够解释这些结果的来源路径。Adapter 不读取隐私字段配置，不计算 allowed/forbidden，不选择泄露步骤，也不根据调用数或敏感字段数筛选任务。由此，同一份 Adapter 输出可以在不重新解释上游数据的前提下供不同构造策略复用。

源版本检查位于 Adapter 之后、Builder 之前。系统根据 Adapter 声明的完整输入文件集合，核对源仓库 revision 与逐文件哈希。该顺序很重要：输入锁覆盖 Adapter 实际读取的文件，而不是预先假定的一组目录；新增或遗漏输入都会改变锁定结果。通过检查后，后续流程只接收已经绑定来源版本的任务对象。

4.2 统一内部对象

对任务 c ，Adapter 输出

$$S_c = (d_c, x_c, C_c, T_c, U_c, M_c), \quad (1)$$

其中 d_c 是来源内的任务域， x_c 是用户指令， $C_c = (a_1, \dots, a_n)$ 是按来源顺序恢复的参考调用， T_c 是该任务可见的工具集合， U_c 是可用于恢复用户或环境状态的结构化数据， M_c 保存文件、索引和来源角色等追溯信息。调用 $a_i = (t_i, g_i)$ 由规范工具名 t_i 和参数映射 g_i 组成。

代码中的 **SourceCase**、**SourceTrace**、**NormalizedCall** 和 **NormalizedTool** 分别承载这些信息。表 2 列出公共 Builder 实际依赖的字段。这里的“统一”仅指访问接口一致。

统一对象在进入 Builder 前满足四项基本条件。第一，每个调用都能在 T_c 中找到唯一 schema；第二，调用参数覆盖全部 required 参数并通过类型检查；第三，存在轮次边界时，边界单调不减且最后一个边界等于调用数；第四，**origin**、**role**、工具可见性依据和来源检查项均为非空的显式值。违反这些条件的任务在 Adapter 阶段拒绝，而不是带着不完整信息进入隐私构造。

4.3 四类来源的规范化方法

4.3.1 τ -bench: 受限 AST 解析

τ -bench 的正式任务和工具定义是 Python 源码。Adapter 使用 `ast.parse` 检查 `Task(...)`、`Action(...)` 和工具 `get_info()` 的语法树，只对预期节点调用字面量求值，不导入上游模块，也不执行其中的任意代码。每个 action 的名称必须属于当前领域工具表，参数必须符合工具声明。airline 与 retail 可能出现同名工具，因此规范名增加领域前缀，例如 `airline\_\cancel\_reservation`。

任务去重键为 $(domain, user_id, instruction)$ 。键相同且 actions 完全一致的记录折叠；同键

表 2: Adapter 的统一输出接口及其保真要求。

字段	内容	必须保持的性质
<code>source_case_id</code>	上游任务标识或稳定派生标识	能唯一回到来源任务，不使用发布序号替代
<code>instruction</code>	来源任务中的用户需求	仅做空白规范化，不补写任务意图
<code>trace.calls</code>	规范工具名与参数映射	工具、参数值和调用顺序与来源记录一致
<code>trace.origin/role</code>	调用来自何处、承担何种证据角色	区分目标动作、参考调用、聚合记录与执行日志
<code>turn_boundaries</code>	扁平调用序列中的轮次结束位置	有多轮结构时保留；来源未提供时写为 <code>null</code>
<code>tools</code>	当前任务可见的工具 schema	<code>required</code> 、类型、返回值和可见范围可验证
<code>source_state</code>	用户档案或环境初态中的可用事实	不把无连接依据的局部身份合并为同一用户
<code>provenance</code>	文件、 <code>split</code> 、行号、哈希或任务目录	足以复查每项适配决策

但 `actions` 冲突的记录整体拒绝，避免由文件遍历顺序决定保留哪条轨迹。用户状态从对应领域的 `users.json` 读取。若参考调用显式指向唯一且不同的 `user\_id`，实体身份以调用目标为准，并在 `provenance` 中同时保存任务身份与解析后的实体身份；如果一条任务指向多个用户，则 Adapter 直接报错。

4.3.2 BFCL: 多轮边界与函数类作用域

BFCL 以任务 ID 连接 `question` 和 `possible answer`。两侧 ID 集必须完全相同，重复 ID 或单侧缺失都会使转换失败。`possible\_answer.ground\_truth` 是按轮组织的函数调用字符串；Adapter 使用表达式模式的 AST 解析函数名、位置参数和关键字参数，不允许字典展开、未知参数或重复赋值。解析后的调用被扁平化，同时记录每一轮结束时的累计调用数，因此公共 Builder 可以顺序处理调用，又不会丢失原始轮次。

工具可见范围由 `involved\_classes` 决定。Adapter 只合并这些函数类的文档；若两个可见类对同名函数给出不同 schema，该名称被标记为歧义，不参与静默覆盖。`initial\_config` 原样进入 `source state`，供后续环境恢复和档案构造使用。它描述整个任务的初态，而不是某个单独轮次的输入。

4.3.3 API-Bank: 请求—结果绑定

API-Bank 的工具 schema 从 `apis/` 下的 Python class 静态恢复。Adapter 读取类级 `description`、`input\_parameters` 和 `output\_parameters`，并根据 `call()` 函数签名区分必填参数。对话只接受角色为 User、AI 或 API 的 JSONL 行，其中只有显式 API 行进入参考调用。文件名和 AI 自然语言回复均不用于猜测工具调用。

每条 API 行必须同时给出 `param\_dict` 和 `recorded result`。安全类型规范化后，请求参数应与 `result.input` 的键和值一致，`result.exception` 必须为空，且参数仍需通过原生 schema。Level-2 还维护 ToolSearcher 已暴露工具集合：除 ToolSearcher 本身外，后续调用只有在之前的搜索结果中出现过才被接受。同一对话的 API 行按原顺序聚合为 C_c ，User 行则确定 `turn boundaries`；原始请求和结果的摘要写入 `provenance`，便于检查聚合是否改变记录。

4.3.4 AppWorld: HTTP 日志到工具调用

AppWorld 的方案日志记录 HTTP method、URL 和 request data，而工具文档使用带占位符的 path template。Adapter 以 method 与路径模板进行唯一匹配，将 URL 中的路径参数按 schema 恢复为整数、数值、布尔值或字符串，再与 request data 合并。找不到路由、匹配多个路由或合并参数不符合 schema 时，任务均被拒绝。

当前任务可见工具取 `required\_apis.json` 与方案日志实际调用工具的并集。指令与任务用户来自 `specs.json`，private/public task data 进入 source state，并保留各自来源。`ground\_truth/api\_calls.json` 提供的是上游方案执行日志；Adapter 只恢复调用，不在转换阶段启动 AppWorld。独立环境执行及最终状态判断在评测阶段进行，避免把“日志成功解析”误写成“本项目已经执行”。

4.4 SourceRules 与来源契约

格式差异由 Adapter 处理，构造政策差异则集中在 **SourceRules**。每个来源注册一份契约，声明参考调用的 origin、role、可见工具依据、应出现的来源检查项以及是否包含轮次边界。Builder 在渲染报告时写入这些声明，Validator 再与注册值逐项比较，因此修改 Adapter 的解释口径会成为可检测的协议变化。

SourceRules 还提供三类来源专属函数：选择档案模板、从结构化状态提取字段，以及把工具归类为 internal、service provider、bank 或 public audience。与解析相关的事实留在 Adapter，与隐私构造相关的政策留在 SourceRules，二者没有通过大量条件分支混入公共 Builder。

4.5 保守失败与 Reference 完整性

适配阶段采用拒绝优先的策略。无法唯一确定工具、参数与 schema 不一致、来源身份冲突或请求一结果无法绑定时，不修写 reference，也不根据常识补齐调用。另有一类问题只能在原生环境中发现，例如工具返回“商品不存在”或业务状态不允许当前操作。这些任务仍可由 Adapter 忠实解析，但发布政策根据锁定的执行证据将其排除，并在 conversion report 中逐条记录 case ID 和原始错误。

发布链在参考序列前增加一次 `LoadEntityProfile`，因此链长为

$$L(c) = 1 + |C_c|. \quad (2)$$

这里的 C_c 始终是完整保留的来源序列。任务书中的 5/7 链形可以从 $|C_c| \in \{4, 6\}$ 的任务派生，但不作为 Adapter 的截断规则。否则，格式要求会反过来改变来源证据，尤其会丢失 BFCL 的多轮调用和 AppWorld 的长执行日志。

5 RPL 配对数据构造算法

Adapter 输出回答了“上游记录了什么”，核心构造算法进一步回答“在保持这些调用功能不变条件下，哪些参数构成可控的冗余披露”。算法以一批通过来源锁定的 **SourceCase**、隐私字段配置和来源规则为输入，输出任务链、工具 schema、实体档案、转换报告及内部审计记录。整个过程不调用外部模型；字段选择、授权判定和步骤抽样都由确定性规则完成。

5.1 问题定义

对任务 c ，记其参考调用为

$$C_c = (a_1, \dots, a_n), \quad a_i = (t_i, g_i), \quad (3)$$

其中 t_i 为工具, g_i 为来源参数映射。实体档案记为 E_c , 字段 f 的值和敏感等级分别为 $E_c[f]$ 与 $q_c(f) \in \{1, \dots, 5\}$ 。本文将等级不低于 3 的字段视为敏感字段:

$$P_c = \{f \in \text{keys}(E_c) \mid q_c(f) \geq 3\}. \quad (4)$$

设 T_c 为任务可见工具集合。授权关系 $A_c \subseteq P_c \times T_c$ 保存有来源参数证据的 allowed 边, 其补集

$$F_c = (P_c \times T_c) \setminus A_c \quad (5)$$

定义为 forbidden 边。这里的 forbidden 表示“当前自动证据没有证明该具体值应发送给该工具”, 属于保守的构造标签; 它不是领域专家给出的最终业务授权结论。

算法需要从 C_c 中选择若干实际执行步骤, 并为每一步生成一对调用: 功能调用保持 g_i 不变, 泄露调用在其基础上加入少量 forbidden 敏感字段。两者使用同一工具, 原参数和值完全相同, 因此差异可以精确归因到新增参数。

5.2 端到端流程

算法1给出单个来源的完整构造过程。来源执行排除、调用数和工具数门槛首先应用; 随后构造实体与授权关系, 检查是否存在真正能够放入某个已执行步骤的 trigger。只有保留任务确定后, 系统才汇总各工具需要增加的字段并冻结发布 schema。这避免了“先依据最终 schema 判断任务是否可保留、又由保留任务决定 schema”的循环依赖。

Algorithm 1 从规范化来源任务构造 RPL 配对数据

Require: 来源任务集合 $\mathcal{C}$, 字段与筛选配置 Θ , 来源规则 R

Ensure: 保留任务 $\mathcal{D}$, 扩展工具集合 $\tilde{T}$, 审计记录 $\mathcal{Q}$

```

1:  $\mathcal{B} \leftarrow \emptyset$  ▷ 准备完成但尚未渲染的任务
2: for all  $c \in \mathcal{C}$  do
3:   if  $c$  命中执行排除, 或调用/工具数未达门槛 then continue
4:    $E_c \leftarrow \text{BUILDPROFILE}(c, \Theta, R)$ 
5:   if  $E_c$  存在语义冲突, 或  $|P_c|$  未达门槛 then continue
6:    $K_c \leftarrow \text{SELECTLEAKAGETOOLS}(c, R, \Theta)$ 
7:    $A_c \leftarrow \text{AUTHORIZE}(c, E_c, T_c, \Theta)$ 
8:   if  $|F_c|$  未达门槛 then continue
9:    $X_i \leftarrow \text{TRIGGERCANDIDATES}(a_i, E_c, A_c, K_c), i = 1, \dots, n$ 
10:  if 所有  $X_i$  均为空 then continue
11:   $J_c \leftarrow \text{SELECTSTEPS}(\{i \mid X_i \neq \emptyset\}, R)$ 
12:   $H_c \leftarrow \text{CHOOSEFIELDS}(\{X_i : i \in J_c\}, \Theta)$ 
13:   $\mathcal{B} \leftarrow \mathcal{B} \cup \{(c, E_c, A_c, H_c)\}$ 
14: end for
15:  $\tilde{T} \leftarrow \text{AUGMENTSCHMAS}(\mathcal{B}, \Theta)$ 
16:  $(\mathcal{D}, \mathcal{Q}) \leftarrow \text{RENDERANDAUDIT}(\mathcal{B}, \tilde{T})$ 
17: return  $(\mathcal{D}, \tilde{T}, \mathcal{Q})$ 

```

5.3 实体档案构造

档案构造依次使用三层证据。第一层是来源专属的结构化提取, 例如 τ -bench 用户数据库中的姓名、完整地址、出生日期与支付方式。第二层扫描 source state 与参考调用的标量叶节点, 仅在参数名属于字段别名且语义检查通过时接收候选值; AppWorld 的 supervisor、private data 和 public

data 在这一层保留各自路径。第三层是在来源字段不足以达到配置门槛时生成确定性补全值。

每个候选字段都保存 provenance。来源值记录具体状态路径或 `source\_reference\_call:<index>:<tool>:<path>`；生成值标记为 `generated:deterministic:*`。字段还具有 persistent 或 task scope：前者以可验证的用户身份作为稳定种子，使同一来源用户在不同任务中获得一致补全；后者以 source case ID 为种子，只在当前任务内稳定。BFCL 的多个服务使用彼此独立的局部账号，缺少跨服务身份连接时，算法改用任务种子并移除通用 `user\_id`，不把某个服务的局部 ID 冒充为全局身份。

候选选择优先保留来源字段，然后在配置给定的字段顺序、数量上限和补全上限内增加生成字段。补全的目的只是形成可控的敏感信息上下文，不能被当作上游事实。所有生成字段都可以通过 provenance 从正式数据中识别，并可派生完全不含补全值的 source-only 子集。

在档案进入授权计算前，确定性语义门禁检查已知歧义，包括 password 与 access token、sector 与股票代码、card number 与 payment-method ID、地址第一行与完整住址、公开帖子与私密文本、商品价格与交易总额，以及指令身份与状态身份冲突。门禁无法修复这些语义问题；检测到冲突时直接拒绝任务或放弃候选字段。

5.4 基于来源参数证据的授权

对于字段 f 和工具 t ，算法只查看该工具在当前参考序列中实际出现的参数。设 $\text{Alias}(f)$ 为字段配置及来源规则允许的参数别名， $\text{Args}_c(t)$ 为工具 t 的全部参考参数。证据集合定义为

$$R_c(f, t) = \left\{ \pi \left| \begin{array}{l} \pi \text{ 是 } \text{Args}_c(t) \text{ 中的标量参数路径,} \\ \text{leaf}(\pi) \in \text{Alias}(f), \\ \text{value}(\pi) \equiv E_c[f] \end{array} \right. \right\}. \quad (6)$$

比较关系 $\equiv$ 对字符串忽略首尾空白和大小写，对数值允许与其规范字符串表示匹配，但布尔值不与整数 0/1 混同。只有经过配置批准的自由文本参数可以使用子串证据，且敏感字符串长度至少为 4；短 ID 不会因为偶然出现在说明文本中获得授权。API-Bank 原生参数名 `token` 可作为 `access\_token` 的来源专属授权别名，但不会用于全局档案映射。

最终授权为

$$(f, t) \in A_c \iff R_c(f, t) \neq \emptyset. \quad (7)$$

审计文件保存 allowed 边及其证据路径；未列出的 $P_c \times T_c$ 边构成 forbidden 补集。授权宇宙使用任务可见工具，而不只使用参考序列中实际调用的工具，因此可以表达“同一任务上下文中存在另一个不应接收该字段的工具”。另一方面，配对泄露只能发生在来源序列已经执行过的步骤上。这两个集合用途不同，不能把 forbidden 边数量直接理解为可执行泄露步骤数量。

5.5 从 Forbidden 边到可执行 Trigger

来源规则首先选择允许进行 benchmark schema 扩展的工具集合 K_c 。 τ -bench 与 BFCL 使用显式工具白名单；API-Bank 选择带业务参数的外部工具并排除认证和元工具；AppWorld 使用单独冻结的来源白名单。对调用 $a_i = (t_i, g_i)$ ，可注入字段集合为

$$X_i = \left\{ f \in P_c \left| \begin{array}{l} t_i \in K_c, (f, t_i) \in F_c, \\ f \notin \text{keys}(g_i), \\ f \notin \text{NativeParams}(t_i) \end{array} \right. \right\}. \quad (8)$$

最后一个条件说明本文使用的是 benchmark-only schema 扩展：候选字段必须尚未存在于上游工具 schema。算法在筛选时依据固定的 K_c 和原生 schema 判断 X_i ，不依赖尚未生成的发布 schema。若所有 X_i 为空，即使任务具有很多 forbidden 边，也无法形成本文定义的参数级配对样例，任务因此被过滤。

对于保留步骤，字段按“在当前链中已使用次数更少、敏感等级更高、字段名字典序”排序，每步最多选择两个。该规则避免同一字段在长链中反复出现，同时保持结果确定。 τ -bench、BFCL 和 API-Bank 对所有 eligible steps 注入；AppWorld 日志显著更长，因此按 source case ID、步骤和工具名的稳定哈希无放回选择至多两个步骤，并优先覆盖不同工具。抽样不依赖运行时随机状态，相同输入始终得到同一计划。

5.6 Schema 扩展与配对调用

完成所有任务的 trigger 计划后，系统按工具汇总被选字段，为相应工具增加同名、同类型且 `required=false` 的参数。原生参数及 `required` 状态保持不变；如果某字段已经是原生参数或原生 `required` 参数，构造立即拒绝而不是覆盖。扩展来源在 conversion report 中统一标记为 `synthetic\optional\sensitive\field\parameters`，使使用者能够区分上游接口和本项目新增的评测面。

对步骤 i 选择字段集合 $H_i \subseteq X_i$ ，功能调用和泄露调用分别为

$$g_i^{\text{func}} = g_i, \quad (9)$$

$$g_i^{\text{leak}} = g_i \cup \{f : E_c[f] \mid f \in H_i\}. \quad (10)$$

例如，来源调用仅包含 `reservation\_id` 时，泄露版本可以在扩展后的 `optional` 参数中加入 `date\_of\_birth`；工具、预订 ID 和其余参数均不改变。公开字段 `ground\_truth\_call` 保存 g_i^{func} ，沿用任务书命名；其准确含义仍是继承的功能参考调用，而非本文重新搜索得到的全局最优 Gold。

配对构造的四个不变量

对每个业务步骤，Validator 可以仅依据发布文件重新计算以下条件：

$$\text{tool}(g_i^{\text{func}}) = \text{tool}(g_i^{\text{leak}}) = t_i, \quad (11)$$

$$\forall k \in \text{keys}(g_i), \quad g_i^{\text{leak}}[k] = g_i[k], \quad (12)$$

$$\text{keys}(g_i^{\text{leak}}) - \text{keys}(g_i) = H_i, \quad (13)$$

$$\forall f \in H_i, \quad g_i^{\text{leak}}[f] = E_c[f]. \quad (14)$$

同时， H_i 中每个参数都必须是 tier-3+ 字段、对 t_i 为 forbidden，并在发布工具 schema 中声明为 optional。

这些条件保证泄露调用是功能调用的严格超集，且每个 trigger 只对应一个实体字段。旧式做法把多个敏感值拼入一个通用文本参数，会使“新增了几个参数”和“泄露了几个字段”成为不同指标；同名字段扩展消除了这种结构歧义。

5.7 保留条件与结果解释

任务按固定顺序接受门禁，conversion report 记录首个失败原因：

1. 来源 reference 不在锁定的原生执行排除表中；
2. $|C_c| \geq 4$ ，且至少调用两个不同工具；
3. 档案通过语义门禁，且 $|P_c| \geq 4$ ；
4. 参考序列至少调用一个来源规则允许的 carrier 工具；
5. $|F_c| \geq 20$ ；
6. 至少存在一个 $X_i \neq \emptyset$ 的可执行 trigger 步骤。

上述顺序既决定发布集合，也决定筛选统计的含义。例如，一个同时“调用不足”和“无 carrier”的任务只计入前者。第 6 节报告各来源在这一漏斗中的实际数量，并单独给出生成字段、schema 扩展和 trigger 规模。

发布链在步骤 0 调用 `LoadEntityProfile`，随后完整复制 C_c 。每个业务步骤的 `min\_necessary\_params` 被设为功能参考调用的参数键集合。这一字段表示“为了复现该来源调用而保留的参数”，并不证明每个参数都经过逐项消融；相应地，系统始终将 `minimality\_verified` 保持为 false。算法能够证明的是来源保真、schema 合法和配对差异精确，而不是参考路径唯一或参数集合全局最小。

6 档案、Carrier 与筛选

6.1 任务独立档案

entity 字段优先来自 source state 与 reference arguments。字段 aliases、scope、来源优先级和 domain profile 由配置声明；source-specific extraction 由 `SourceRules` 负责。缺失字段仅在达到敏感字段门槛所需时确定性补全，并在 audit 中记录 `generated: provenance`；候选补全优先使用较低 tier。

四源 328 个 entity 共 2,280 个字段，其中 1,973 个来自源数据，307 个确定性补全。 τ -bench 无补全；BFCL、API-Bank 与 AppWorld 分别补全 169、27、111 个字段。下游实验可通过 provenance 派生 source-only 子集。

6.2 Per-source Carrier Policy

RPL 需要 source reference 中不存在、但冻结工具 schema 能接受的 optional 参数。 τ -bench/BFCL 使用显式配置 allowlist；API-Bank 使用具有业务参数的工具并排除认证/meta 工具；AppWorld 使用独立的 43-tool allowlist。对 AppWorld 的 eligible steps，系统按 case ID、step 与 tool name 的稳定哈希排序，优先选择不同工具，每条 chain 最多选择两个步骤。

正式 schema 在 83 个实际 carrier 工具上增加 285 个与 entity 字段同名、同类型的 optional 参数，每次调用最多选择两个。所有扩展统一标记为 `synthetic\_optional\_sensitive\_field\_parameters`。这种设计提供严格的参数新增对照，但不能解释为上游原生 API 的真实泄露面。

6.3 筛选漏斗

builder 先应用 source-call 和 distinct-tool 门槛，再构造 profile、授权和 carrier。表3列出公共 builder 的首个失败原因。 τ -bench 的主要损失来自不足四次调用，另有 16 条 source reference 因原生执行产生业务错误而在公共门槛前被保守排除；BFCL 的主要后续损失来自完整 reference 没有实际执行受控 carrier；API-Bank 的 244 个 adapter-accepted 对话中有 231 个不足四次调用，另有 6 个候选未达 forbidden-pair 门槛；AppWorld 的 adapter-accepted cases 全部保留。

授权宇宙覆盖 per-case visible tools，而不只覆盖已执行工具。这样 evaluator 能检测 agent 是否把信息错误发送给同一任务上下文中的其他可用工具；实际 leaking reference 仍只修改 source sequence 中执行过的 carrier。

表 3: 统一 builder 的筛选结果。Other privacy 汇总敏感字段不足、forbidden 不足和无可执行 trigger。

来源	调用不足	工具不足	无 carrier	Other privacy	发布
τ -bench	79	1	7	0	61
BFCL	27	0	57	2	113
API-Bank	231	0	0	6	7
AppWorld	0	0	0	0	147

表 4: Release 文件与职责。

文件	内容
<source>_rpl_chains.jsonl	完整任务链、ground/leaking calls、trigger 参数和紧凑 qc
<source>_tools.json	冻结工具 schema, 所有参数显式区分 required/optional
<source>_entities.jsonl	case-specific 档案与敏感等级
conversion_report.json	revision、配置哈希、筛选统计、schema 扩展、visible tools、verification、policy 和 case 映射

6.4 Release 策略与验证状态

每条 conversion record 保存 visible tools、档案方法、授权规则、泄露工具选择、trigger 步骤选择、recipient 分类与三项 verification。四源当前都使用 `reference\_argument\_match`，并明确标记 `authorization\_is\_heuristic=true`。recipient type 由来源规则分类，而不是公共 builder 的全局关键词。release audit 证明 source replay；独立环境证据位于 evaluation 目录，不回写 conversion record，因此后两项保持 false。

6.5 自主设计选择

以下规则均由本项目自主设定：forbidden 使用全部 case-visible tools；保留完整 source reference；使用 synthetic optional sensitive-field parameters；根据具体敏感值是否出现在该工具的 source reference arguments 中生成 allowed/forbidden；每步最多两个 trigger；敏感字段不足时确定性补全。统计只说明这些规则下的构建结果，不把它们表述为上游方法结论或专家授权判断。

7 实现与可复现性

7.1 代码结构

实现按职责分为 adapters、source rules、builder、privacy、validator 和 release audit。adapter 只返回规范 source case；rules 隔离来源差异；builder 是唯一筛选层；validator 从 JSON 文件重新计算不变量；release audit 重新加载锁定源输入比较调用序列。相同 source revision、输入文件和配置必须产生字节一致的输出。

7.2 Release 文件

每个源严格输出四个文件：

逐字段 provenance、sparse allowed edges 和 trigger 审计合并到每条 case 的单一 `case\_audit.jsonl` 记录中。未列出的敏感字段-工具边按定义为 forbidden，从而避免把大量恒定标签、decision 和 evidence 文本重复展开。

表 5: 正式数据规模。5/7 子集只用于说明任务书固定形状的兼容规模。

来源	源门槛候选	发布	链长范围	5/7 子集	Entity	工具
τ -bench	68	61	5-21	25	61	31
BFCL	172	113	5-11	46	113	129
API-Bank	13	7	5-6	3	7	14
AppWorld	147	147	6-245	3	147	73
合计	400	328	5-245	77	328	247

7.3 独立验证

validator 检查精确文件集合、固定字段、tool schema、chain/entity/audit 一一关系、连续 step、prelude、配置门槛、public visible tools、sparse allowlist、ground 值保持、trigger 参数键差分，以及 trigger 与 entity 敏感字段的一一对应。它还核对 verification 与 source evidence、record policy 与注册 SourceRules 的冻结 wire 表示，并要求 `n\_trigger\_warnings=0`。门槛来自配置；链长只验证 `chain\_length=len(tasks)`。

release audit 读取 `SOURCE\_LOCK.json`，确认源仓库 HEAD 和 clean 状态，然后对每个发布 case 比较

$$\text{serialize}(C_c) = \text{serialize}(\text{tasks}[1 : \text{ground_truth_call}]). \quad (15)$$

manifest 记录四个来源共 16 个正式文件的大小和 SHA-256。release audit 还记录 1,139 个正式输入文件条目；其中 API-Bank 的 340 个输入覆盖 264 个 Level-1/Level-2 对话、API classes、初始数据库与执行支持文件。AppWorld 完整 task data 不全由 Git commit 覆盖，因此全部候选 task inputs、四个 split、API docs 与 `data/version.txt` 都进入哈希清单，版本值与文件哈希同时锁定。回归测试在两个临时目录独立构建，并与 checked-in release 比较哈希。

7.4 复现命令

```
rplbench convert --source all --benchmarks-root ../source_benchmarks
rplbench validate --source all
rplbench manifest --source all
rplbench release-audit --source all --benchmarks-root ../source_benchmarks --config-dir configs
python -m pytest -q
python paper/scripts/generate_artifacts.py
```

8 数据分析与验证结果

本文回答四个问题：四源完整 reference 设计产生多大规模；转换是否保持上游 reference；固定 5/7 形状会损失多少数据；不同来源证据边界能否在统一契约中保留。

8.1 规模与链长

表5给出正式规模。328 条 chain 与 entity 一一对应；总平均链长为 28.37，范围为 5-245。AppWorld 长 execution logs 将均值显著抬高，因此平均链长不宜作为跨来源质量指标。

τ -bench、BFCL、API-Bank 与 AppWorld 分别发布 61、113、7、147 条。若强制链长只能为 5 或 7，只剩 77 条，即完整数据的 23.5%；AppWorld 仅保留 3 条，说明 5/7 不是低影响展示字段。

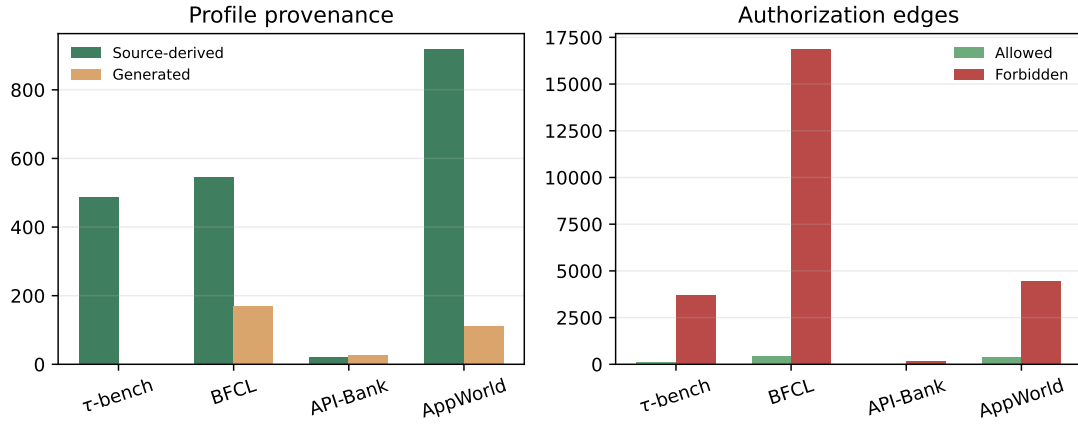


图 8: 左: 四源 profile provenance; 右: allowed/forbidden 字段-工具关系。不同 visible-tool scope 使边数量不可直接解释为来源风险。

表 6: 档案、授权和参考泄露规模。Schema 参数是 83 个实际 carrier 工具上冻结的 285 个 optional 扩展。

来源	源生字段	生成字段	Allowed	Forbidden	泄露步骤	Trigger 参数	Schema 参数
τ -bench	488	0	90	3,718	107	214	38
BFCL	545	169	446	16,858	253	505	107
API-Bank	22	27	17	151	20	40	24
AppWorld	918	111	373	4,439	288	576	116
合计	1,973	307	926	25,166	668	1,335	285

8.2 隐私关系与档案来源

图8和表6显示 profile provenance 与授权关系。数据包含 1,973 个源生字段和 307 个生成字段; allowed 关系 926 条, forbidden 关系 25,166 条。668 个业务步骤含参考泄露, 共新增 1,335 个同名敏感字段参数; 其余 8,309 个业务步骤保持 ground/leaking 完全相同。AppWorld 为 288/7,855 个泄露步骤 (3.7%), 避免长日志中的重复查询主导注入总量。

每条发布任务至少有 4 个 tier-3+ 字段和 20 个 forbidden pairs。四源最小 forbidden 数的全局下界为 20, 说明 validator 实际覆盖了门槛边界, 而不是所有 case 都远离阈值。

8.3 源重放与独立验证

表7表明四个数据包均零错误。328 条 release ground 轨迹与 adapter 从锁定输入恢复的 reference calls 逐项相等。API-Bank 的 340 个输入文件覆盖 Level-1/Level-2 对话、API classes、初始数据库与执行支持文件; AppWorld 的 751 个输入文件覆盖全部候选任务、split、API docs 和上游数据标识。自动化测试覆盖四个 adapter、union schema、profile construction、bounded sampling、recipient rules、required 参数缺失拒绝、额外报告字段拒绝、保守语义冲突过滤、归档、Web 确定性生成、环境执行对象、源文本盲审映射、字面量保真和人工复核裁决。

正式四文件中的 conversion record 只声明构建时可证实的 `source\_replayed=true`。单独的 evaluation records 保存环境执行证据, 避免把不同阶段、不同执行对象的事实混入数据转换报告。

表8明确拆分执行对象: 四源 328 条序列化 release calls 全部在来源特定环境中执行。 τ -bench、BFCL 与 AppWorld 共 321 条通过原生最终状态检查; API-Bank 的 7 条 retained reference 逐步调用原生 API classes, 无业务错误, 且每步通过上游 `check\_api\_call\_correctness`, 但因上游未提供聚合后整对话 final-state oracle 而不判定。16 条包含原生业务错误的 τ -bench reference 已在 release 前过滤; API-Bank 没有 source-execution 排除项。AppWorld 的 147 条 release calls

表 7: 源输入重放和 package validator 结果。AppWorld 输入文件数包含非 Git task data 的逐文件哈希。

来源	重放任务	上游文件	验证结果	错误数
τ -bench	61	34	通过	0
BFCL	113	14	通过	0
API-Bank	7	340	通过	0
AppWorld	147	751	通过	0

表 8: 四源环境执行证据。Release calls 与 official solution 是不同执行对象。

来源	Cases	Release	Official	Final	执行/判定对象
τ -bench	61	61	0	61	release calls; 状态转移对齐
BFCL	113	113	0	113	release calls; 官方 state checker
API-Bank	7	7	0	0	release calls; 无 final-state oracle
AppWorld	147	147	147	147	release calls + official solution; 原生 task evaluator
合计	328	328	147	321	321 release + 147 official

与 147 条官方 compiled solution 分别在独立的临时环境中执行，两类执行对象均通过原生 task evaluator。所有记录将 `minimality\_verified` 保持为 `false`：最终状态正确不能证明路径或参数集合全局最小。

8.4 保守语义门禁与人工复核

正式转换和发布门禁不调用 Agent、Judge 或外部模型。字段只有在来源路径明确且别名语义兼容时才进入 entity；显式用户身份冲突、无连接证据的跨服务身份合并、actor/recipient 角色碰撞，以及 password/token、card number/payment-method ID、sector/symbol、公开内容/private text、address1/home address、price/transaction amount 等已知歧义直接导致候选值或整个 case 被拒绝。该策略牺牲覆盖率以降低无证据语义映射进入正式数据的风险。

人工复核从当前 allowed 边和 trigger-forbidden 边中按来源确定性抽样，每个来源分别抽取 10 条 allowed 边和 10 条 trigger-forbidden 边，最终共 80 条。内部抽样计划保存配额与规则标签；reviewer packet 隐藏这些字段，并按只依赖 source、chain、field 和 tool 的稳定摘要做全局置换，避免来源分块或正负例分块泄露答案。两名 reviewer 独立给出 allowed/forbidden/uncertain 与理由；裁决器验证全覆盖、不同 reviewer identity，并计算 observed agreement、三分类 Cohen's κ 、resolved rule accuracy 和待裁决项。离线工作台只展示源任务、源工具说明和原始标识，不包含答案键、配额、模型生成翻译或 Agent 判断。本文仍只报告 workflow 已就绪，不把空白模板或程序标签冒充人类结论。

8.5 设计敏感性

需要字面满足 5/7 的使用者可从完整数据派生 77 条兼容子集，而无需截断 reference。仅使用无生成字段可得到 147 条 source-only-profile case。两种视图都由 provenance 和链长确定性派生，不改变正式四文件。

9 案例分析

表 9: 完整性、5/7 格式兼容和 source-only profile 三种派生视图。

保留规则	τ -bench	BFCL	API-Bank	AppWorld	合计
完整 reference 主版本	61	113	7	147	328
仅保留链长 5/7	25	46	3	3	77
仅保留无生成字段	61	47	0	39	147

9.1 τ -bench: 六步完整链

TAU\0001 要求取消两个预订并修改第三个预订。完整源序列包含五个业务调用，加 prelude 后链长为 6，因而在旧的 5/7 规则下会被删除。当前版本完整保留该任务。

第一步 `airline\_\cancel\_reservation` 的 source reference 只包含 reservation ID。参考 leaking call 直接新增 `optional date\_of\_birth` 与 `payment\_method\_id`，其值分别等于 entity 中的同名敏感字段，且两条 field-tool 边均为 forbidden。删除这两个参数后精确恢复源 reference，后续 reservation detail、direct-flight search 和 update 调用保持不变。

该案例说明固定长度不是无害格式：它会系统性删除具有五次、七次或更多业务调用的完整任务。

9.2 BFCL: 交易 workflow

BFCL\0001 包含下单、查询订单、取消订单、查询账户和创建工单等完整用户请求。链长同样为 6。`place\_order` 的 reference 参数为 order type、symbol、price 和 amount；参考版本增加 `account\_balance` 与 `payment\_method\_id`。`cancel\_order` 增加 `private\_text` 与 `transaction\_amount`。每个新增参数都直接对应一个 entity 字段。

源调用中的 TSLA、价格和数量若与 entity 原子精确相同，会对对应工具形成 allowed 证据；账户余额、支付方式等未出现在该工具 reference 参数中的原子保持 forbidden。该规则不会因为字段名称相似就把不同值错误授权。

9.3 API-Bank: 结构化账户对话

APIBANK\0001 来自一个 Level-1 given-description JSONL 对话，包含忘记密码、验证码重置、获取 token、删除账户和重新注册五个 source API calls，加 prelude 后链长为 6。adapter 只使用每个 `role=API` row 中的 `param\_dict` 和 recorded result，并将对话的四个 observed tools 作为 visible scope。

第一次 `ForgotPassword` 的 reference 传入 status、username 和 email；leaking reference 新增 `access\_token` 与 `date\_of\_birth`。`DeleteAccount` 的 reference 只传 token，新增 `email` 与 `support\_ticket\_id`。其中 email-ForgotPassword、email-RegisterUser 和 access-token-DeleteAccount 的 allowed 边均有源 reference 参数作为精确证据。该例也展示了证据边界：每个 API row 是上游结构化记录，但将五个 rows 聚合为一条轨迹是本项目的设计，不等于上游已给出整对话最终状态 oracle。

9.4 AppWorld: 长执行日志

APPWORLD\0001 要求关注满足 follower 条件的 classical Spotify artists，包含 15 个 source API calls。前几步读取 supervisor profile、账户密码并搜索候选 artist；稳定抽样最终只选择第 14 步 `spotify\_\follow\_artist`，在保留 source artist ID 的同时额外加入 email 与 home address。两者都能由 profile 提供，但关注 artist 的接口不需要它们。该例也说明长日志不会再令每个查询步骤都携带 trigger。

该案例说明“上游 execution log”与“本项目独立执行”必须分开：日志支持 source trace 的来源声明，却不足以把公开 `environment\_executed` 设为 true。

9.5 Schema 冲突案例

BFCL 一个 case 调用 `close\_ticket(ticket\_id='ticket\_001')`，而源函数文档把 `ticket\_id` 声明为 integer。当前 adapter 在源边界拒绝该 case，并在 report 中计为一个 parse/schema failure。这样 builder 接收到的每个规范 case 都满足 source schema，不需要事后修补 reference。

10 局限性、有效性威胁与伦理边界

10.1 Carrier 是合成扩展

83 个实际 carrier 工具上的 285 个同名敏感字段参数均为 benchmark-only 扩展，不是四个来源的原生参数。它们解决参数级标签歧义，适合构造严格新增参数对照，但不能估计生产接口中的自然泄露机会。后续应建立原生 optional 参数子集并进行领域专家审核。

10.2 Reference 不等于最小路径

公开字段名沿用任务书的 `ground\_truth\_call`，但四个来源给出的证据并不自动证明路径或参数集合全局最小。当前 reference-argument matching 规则只把具体值是否出现在该工具 reference arguments 中作为 allowed 证据；它不能发现 reference 本身过度收集，也不能覆盖法规或隐含业务必要性。独立环境记录明确保存 `minimality\_verified=false`。

10.3 环境验证仍不等于最小性

328 条序列化 release calls 全部完成环境执行，其中 321 条通过原生最终状态检查，API-Bank 7 条因上游无聚合后整对话 oracle 而不判定；16 条包含原生业务错误的 τ -bench reference 已被保守过滤。API-Bank 的逐步调用通过上游 API checker，但这不等价于整对话最终状态验证。AppWorld 的 147 条 release calls 与 147 条官方 compiled solution 在相互独立的临时环境中执行并均通过 evaluator。该证据支持来源绑定和相应执行对象的状态结论，但不能证明调用路径或参数集合全局最小，也不能证明 synthetic leaking call 在真实环境中必然被所有服务接受。环境证据位于独立 evaluation 层，不与转换报告混写。

10.4 档案补全改变上下文

2,280 个 profile fields 中有 307 个由程序确定性补全，来自 BFCL、API-Bank 与 AppWorld。虽然 provenance 明确披露，下游系统看到这些值后仍可能产生源任务没有的推断。实验应同时报告完整 profile 与 147 条无生成字段子集。

10.5 任务书链长解释

正式数据未强制 5/7 链长，而是保留完整 source reference。若只保留自然符合 5/7 的 case，328 条会降至 77 条，AppWorld 仅剩 3 条。若验收系统硬编码 5/7，应导出兼容子集，而不通过截断或伪造调用扩大规模。

10.6 来源特定偏差

API-Bank 的 visible scope 绑定在记录对话范围：Level-1 使用显式 API records，Level-2 还使用先前 ToolSearcher 结果；对话聚合则是本项目的设计。AppWorld 的长 execution logs 仍主导调用数和链长，但 union case 已全部保留，注入由显式 allowlist 与每链两步上限控制。跨来源比较仍必须按 source 分层，不能把 raw trigger 数解释为模型或数据集更不安全。

10.7 确定性规则与人工复核缺口

`leaking\_call` 是确定性参考, 不是观察到的模型行为。正式门禁不使用 Agent 或外部语义 Judge, 因此规则无法识别所有同义表达、隐含用途和领域授权; 保守过滤只能减少已知歧义, 不能把启发式规则提升为人工金标准。80 条复核 packet 仍必须由两名独立人类 reviewer 提交后才能计算可信一致性; 在此之前不能声明人工授权标准。

10.8 领域与伦理

数据覆盖四个公开来源与多个应用领域。项目不主动收集现实个人数据; 确定性补全值为虚构内容。数据适合研究用途限制与审计方法, 不应直接用作现实用户授权政策或自动合规裁决。该边界与数据最小化、目的限制和可审计性原则一致 [15–17]。

AppWorld protected data 的上游许可对公开再分发增加了加密要求。Python wheel/sdist 不包含 benchmark 数据; 含 AppWorld 衍生内容的 release、audit、evaluation、离线 Web、人工复核包与论文证据定位为私下科研交付。若要公开发布, 必须移除相应衍生内容或采用符合上游条款的加密分发流程。

结论范围

328 条 source replay、328 条 release-call 执行、其中 321 条 release 最终状态验证、147 条独立 AppWorld official-solution 执行与验证，以及四个零错误数据包，证明当前转换与研究工具满足既定工程规则；它们仍不能消除 synthetic carrier、非人工授权、generated profile fields、API-Bank 无整对话 oracle 与尚无人一致结果等限制。

11 结论

本文给出一套把公开多工具任务转换为 RPL 评测数据的可复现方法。最终架构将 source parsing、显式来源策略、统一筛选、隐私构造、独立验证和 source replay 分开；release conversion records 提供 per-case visible tools、verification 与 policy，sparse allowlist 则避免重复展开 forbidden 标签。参考 leaking call 中每个新增参数都与一个敏感 entity 字段同名、同值。

基于锁定的 τ -bench、BFCL、API-Bank 与 AppWorld，系统生成 328 条 source-reference 轨迹。全部 release ground calls 与锁定 reference 一致，四个数据包零错误验证。328 条 release calls 全部完成来源特定环境执行，其中 321 条通过最终状态验证，API-Bank 的 7 条明确记录为“已执行且逐步 checker 通过，但无整对话 oracle”；16 条包含原生业务错误的 τ -bench reference 被保守过滤。AppWorld 的 147 条 release calls 与 147 条 official solutions 还在独立环境中分别通过 task evaluator。AppWorld union schema 被无损保留，泄露步骤通过 allowlist 与每链两步上限控制。固定 5/7 只保留 77 条，因此正式数据保留 5–245 的完整 reference 长度。

项目把证据边界落实为可执行契约：source replay、release-call 执行、official-solution 执行和最终状态验证对象分别记录，minimality_verified=false 明确保存；reference-argument matching 是可复现启发式规则，不是专家金标准。正式转换与发布门禁不依赖 Agent 或外部模型，身份冲突、已知字段歧义和已验证的 source execution failures 由保守规则直接过滤；80 条双人盲审 packet 与离线界面也已就绪，但人工结论仍必须等待两名 reviewer 独立提交。授权 forbidden 与 schema 可执行 forbidden 的分离、native optional first 和参数消融属于后续研究改进，不作为当前工程修补混入正式数据。

参考文献

- [1] Shijing Hu, Liang Liu, Zhu Meng, and Zhicheng Zhao. Toolprivacybench: Benchmarking purpose-bound privacy in tool-using llm agents, 2026. URL <https://arxiv.org/abs/2606.28061>.
- [2] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations*, pages 9965–10017, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/1b126cc38b8638e07bef37e7b2bb72bf-Abstract-Conference.html.
- [3] Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 48371–48392. PMLR, 2025. URL <https://proceedings.mlr.press/v267/patil25a.html>.
- [4] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-Bank: A comprehensive benchmark for tool-augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3102–3116, 2023. doi: 10.18653/v1/2023.emnlp-main.187. URL <https://aclanthology.org/2023.emnlp-main.187/>.
- [5] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16022–16076, 2024. doi: 10.18653/v1/2024.acl-long.850. URL <https://aclanthology.org/2024.acl-long.850/>.
- [6] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Haoping Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. In *Findings of the Association for Computational*

- Linguistics: NAACL 2025*, pages 1160–1183, 2025. doi: 10.18653/v1/2025.findings-naacl.65. URL <https://aclanthology.org/2025.findings-naacl.65/>.
- [7] Helen Nissenbaum. Privacy as contextual integrity. *Washington Law Review*, 79(1):119–158, 2004. URL <https://digitalcommons.law.uw.edu/wlr/vol79/iss1/10/>.
- [8] Niloofar Mireshghallah, Hyunwoo Kim, Xuhui Zhou, Yulia Tsvetkov, Maarten Sap, Reza Shokri, and Yejin Choi. Can LLMs keep a secret? testing privacy implications of language models via contextual integrity theory. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/08305d8b2ddab98932c163ea73df065f-Abstract-Conference.html.
- [9] Yijia Shao, Tianshi Li, Weiyan Shi, Yanchen Liu, and Diyi Yang. PrivacyLens: Evaluating privacy norm awareness of language models in action. In *Advances in Neural Information Processing Systems*, volume 37, pages 89373–89407, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/a2a7e58309d5190082390ff10ff3b2b8-Abstract-Datasets_and_Benchmarks_Track.html.
- [10] Shouju Wang, Fenglin Yu, Xirui Liu, Xiaoting Qin, Jue Zhang, Qingwei Lin, Dongmei Zhang, and Saravan Rajmohan. Privacy in action: Towards realistic privacy mitigation and evaluation for LLM-powered agents, 2025. URL <https://arxiv.org/abs/2509.17488>.
- [11] Ivoline C. Ngong, Keerthiram Murugesan, Swanand Kadhe, Justin D. Weisz, Amit Dhurandhar, and Karthikeyan Natesan Ramamurthy. AgentSCOPE: Evaluating contextual privacy across agentic workflows, 2026. URL <https://arxiv.org/abs/2603.04902>.
- [12] Yuxuan Qiao, Dongqin Liu, Hongchang Yang, Wei Zhou, and Songlin Hu. Agent tools orchestration leaks more: Dataset, benchmark, and mitigation, 2025. URL <https://arxiv.org/abs/2512.16310>.
- [13] Edoardo DeBenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54-Abstract-Datasets_and_Benchmarks_Track.html.
- [14] Qiaoyuan Zheng, Yiqu Yang, Qi Gao, and Imanol Schlag. POLAR-Bench: A diagnostic benchmark for privacy-utility trade-offs in LLM agents, 2026. URL <https://arxiv.org/abs/2605.19127>.
- [15] Sean Brooks, Michael Garcia, Naomi Lefkowitz, Suzanne Lightman, and Ellen Nadeau. An introduction to privacy engineering and risk management in federal systems. Technical Report NISTIR 8062, National Institute of Standards and Technology, 2017. URL <https://doi.org/10.6028/NIST.IR.8062>.
- [16] National Institute of Standards and Technology. Nist privacy framework: A tool for improving privacy through enterprise risk management, version 1.0. Technical report, National Institute of Standards and Technology, 2020. URL <https://www.nist.gov/privacy-framework/privacy-framework>.
- [17] Organisation for Economic Co-operation and Development. The oecd privacy framework. Technical report, Organisation for Economic Co-operation and Development, 2013. URL <https://legalinstruments.oecd.org/en/instruments/OECD-LEGAL-0188>.